

Project: Secure File Sharing with S3 and CloudFront

Services Used

Amazon S3, Amazon CloudFront, IAM (Bucket Policy), CloudFront Key Groups, Signed URLs, (Optional: Lambda@Edge)

Description

This project implements a secure file-sharing system using AWS where files are stored privately in Amazon S3 and delivered to users through Amazon CloudFront using signed URLs. Direct access to S3 is blocked, and only authorized users with a temporary signed URL can access the files.

Definition

Secure file sharing using CloudFront and S3 is a mechanism where private files are distributed through a CDN using temporary, signed URLs that restrict access based on time and authorization.

Scope of the Project

- Create a private S3 bucket for file storage
- Upload files securely
- Configure CloudFront for content delivery
- Use Origin Access Control (OAC) for secure S3 access
- Apply IAM bucket policy to restrict direct access
- Generate signed URLs for temporary file access
- Validate secure access behavior

This Project Covers

- Secure S3 configuration
- CDN setup using CloudFront
- Origin Access Control (OAC)
- IAM resource-based policies
- Public and private key usage
- Signed URL generation
- Temporary secure file access

Methodology

1. Secure File Delivery Architecture (Implemented Method)

- Files stored in private S3 bucket
- CloudFront acts as CDN layer
- OAC restricts direct S3 access
- Signed URLs control user access
- Only authorized temporary access is allowed

2. Public File Hosting (Alternative Method)

- Files can be made public in S3
- No CloudFront restriction
- No security
- Not used due to lack of access control

AWS Services Used in the Project

1. Amazon S3

Definition: Object storage service for storing files.

Purpose: Store files securely.

Role: Hosts private files that should not be directly accessible.

2. Amazon CloudFront

Definition: Content Delivery Network (CDN) service.

Purpose: Deliver files globally with low latency.

Role: Acts as the only access point to S3 files.

3. IAM (Bucket Policy)

Definition: AWS permission system.

Purpose: Control access to S3 bucket.

Role: Allows only CloudFront service to access S3.

4. CloudFront Signed URLs

Definition: Temporary secure URLs with expiration.

Purpose: Provide controlled access to private files.

Role: Ensures only users with valid links can access files.

5. Key Groups & Public Keys

Definition: CloudFront trusted key mechanism.

Purpose: Validate signed URLs.

Role: Stores public key used to verify requests.

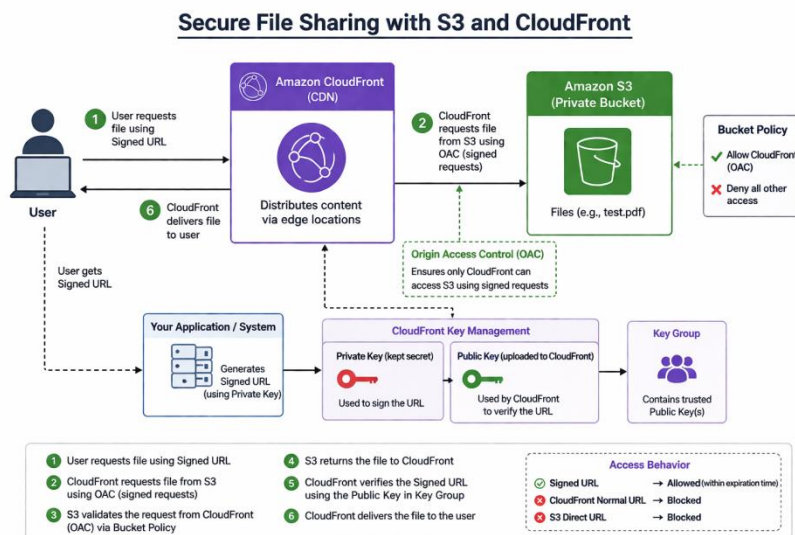
6. Lambda@Edge (Optional)

Definition: Serverless function at CDN edge locations.

Purpose: Add custom authentication logic.

Role: Can validate tokens, headers, or user identity.

Architecture:

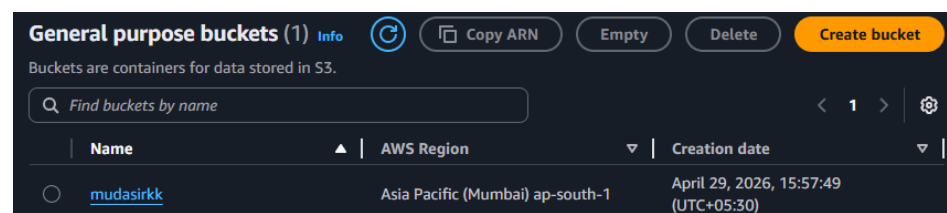


Detailed Steps per AWS Service

Amazon S3 (Storage Setup)

Step 1: Create S3 Bucket

- Go to AWS Console → S3 → Create bucket
- Bucket name: secure-file-sharing-mudasir
- Region: ap-south-1
- Block all public access: **Enabled**



Purpose:

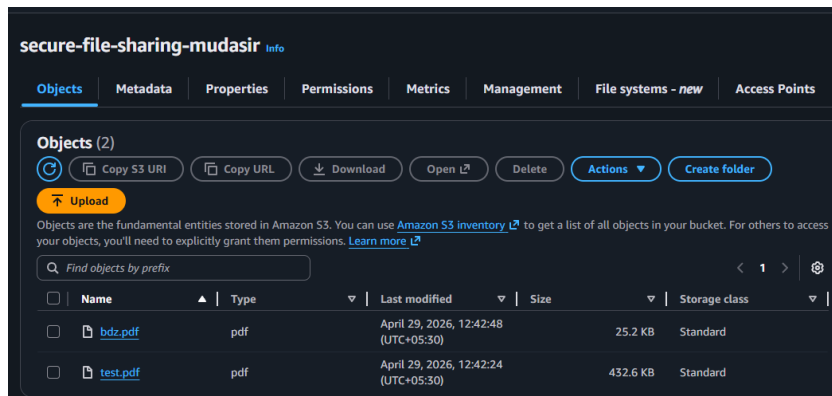
S3 bucket is created to store files securely and prevent direct public access.

Step 2: Upload Files

- Open bucket → Upload
- Upload file (e.g., test.pdf)
- Rename file (avoid spaces)

Purpose:

Files are uploaded to S3 for secure distribution via CloudFront.



Step 3: Verify Private Access

- Open S3 object URL

Expected: Access Denied

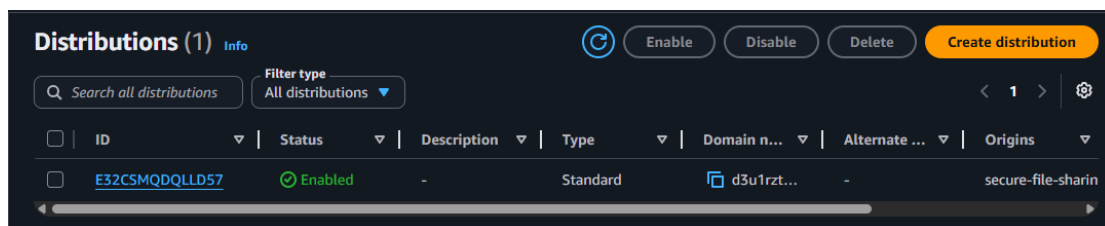
Purpose: Ensures that files are not publicly accessible.

Step 4: (CDN Setup) Create CloudFront Distribution

- Go to CloudFront → Create distribution
- Origin: Select S3 bucket
- Origin access: Create new OAC
- Viewer protocol: Redirect HTTP to HTTPS
- Cache policy: CachingOptimized

Purpose:

CloudFront is used to deliver files globally with low latency.



Step 5: Manually Configure Origin Access Control (OAC)

- Select: Sign requests (recommended)
- Save OAC

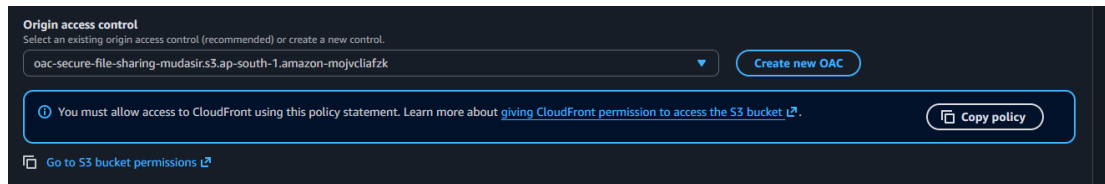
Purpose:

OAC ensures CloudFront securely accesses S3 without making bucket public.

IAM (Bucket Policy)

Step 6: Add Bucket Policy

- Go to S3 → Permissions → Bucket policy
- Paste CloudFront-generated policy



Purpose:

- Allow CloudFront access
- Block direct public access

CloudFront Public Key & Key Group

Step 7: Generate RSA Key Pair

- Use OpenSSL or online generator
- Generate:

- Private key (.pem)
- Public key

Purpose:

Keys are used for signed URL authentication.

RSA Key Generator

You may generate an RSA private key with the help of this tool. Additionally, it will display the public key of a generated or pasted private key.

Key Length

2048

Generate key pair

Private key	Public key
-----BEGIN RSA PRIVATE KEY----- MIIEogIBAAKCAQ89Q12WozJF3kUXG3yg5Q3nYZCjvjrFb1wEfz21Y+DY+RhKJD D3zrC4d1ubtCUIB8-066(b1J7UKTb-7T03c1066Hm-6elD3r89Eh48UDn7ocwV -----	-----BEGIN PUBLIC KEY----- MIIBITANBgkqhkiG9w0BAQEFAAOCAQ4AMIIBCQKCAQ89Q12WozJF3kUXG3yg5Q3n Y7C4zrC4d1ubtCUIB8-066(b1J7UKTb-7T03c1066Hm-6elD3r89Eh48UDn7ocwV -----

Step 8: Upload Public Key

- Go to CloudFront → Public keys → Create
- Paste public key

Purpose:

CloudFront uses this key to verify signed URLs.

Public keys				Edit	Delete	Create public key
Search public keys						
☐	ID	Name	Type	Description		
☐	KWXNAFORK6JIE	public-key	RSA 2048			

Step 9: Create Key Group

- Go to CloudFront → Key groups → Create
- Add public key

Purpose:

Defines trusted keys allowed to access content.

Cache statistics
Popular objects
Top referrers
Usage
Viewers
▼ Security
Origin access
Trust stores
Field-level encryption
▼ Key management
Public keys
Key groups
▼ Savings Bundle

Key group

Name
Enter a name for this key group.

Description - optional
Enter a description for this key group.

Public keys
Choose from 1 to 5 public keys to add to this key group.

public-key
KWXNAFORK6JIE RSA 2048

CloudFront Security Configuration

Step 10: Restrict Viewer Access

- Go to CloudFront → Behavior → Edit
- Enable:

Restrict viewer access = Yes

- Select trusted key group

Purpose:

Blocks normal access and allows only signed URLs.

General	Security	Origins	Behaviors	Error pages	Invalidations	Logging	Tags																
Behaviors (1/1) Filter behaviors by property or value Save Move up Move down Edit Delete Create behavior <table> <tr> <th></th><th>Preced...</th><th>Path pattern</th><th>Origin or origin group</th><th>Viewer protocol policy</th><th>Cache policy name</th><th>Origin request polic...</th><th>Response headers p...</th></tr> <tr> <td>0</td><td></td><td>Default (*)</td><td>secure-file-sharing-...</td><td>Redirect HTTP to HT...</td><td>Managed-CachingOptimiz</td><td>-</td><td>-</td></tr> </table>									Preced...	Path pattern	Origin or origin group	Viewer protocol policy	Cache policy name	Origin request polic...	Response headers p...	0		Default (*)	secure-file-sharing-...	Redirect HTTP to HT...	Managed-CachingOptimiz	-	-
	Preced...	Path pattern	Origin or origin group	Viewer protocol policy	Cache policy name	Origin request polic...	Response headers p...																
0		Default (*)	secure-file-sharing-...	Redirect HTTP to HT...	Managed-CachingOptimiz	-	-																

Step 11: Generate Signed URL (Python):

Created a Python script using CloudFrontSigner to generate a signed URL using the private key.

Steps:

- Added Key ID from CloudFront
- Provided CloudFront file URL
- Used private_key.pem for signing
- Set expiration time (10 minutes)

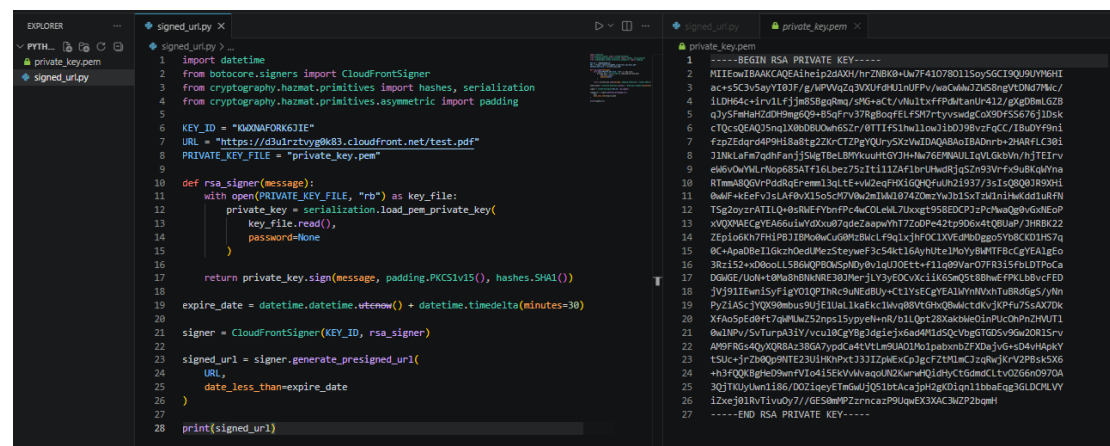
Purpose:

Generate a temporary secure URL that allows controlled access to private content.

Add Private Key

- Save private_key.pem in same folder

Purpose: Private key is used to sign the URL.



```
1 import datetime
2 from boto3.signers import CloudFrontSigner
3 from cryptography.hazmat.primitives import hashes, serialization
4 from cryptography.hazmat.primitives.asymmetric import padding
5
6 KEY_ID = "K3WNAFORK6JIE"
7 URL = "https://d3u1rztvyg0k83.cloudfront.net/test.pdf"
8 PRIVATE_KEY_FILE = "private_key.pem"
9
10 def rsa_signer(message):
11     with open(PRIVATE_KEY_FILE, "rb") as key_file:
12         private_key = serialization.load_pem_private_key(
13             key_file.read(),
14             password=None,
15         )
16
17     return private_key.sign(message, padding.PKCS1v15(), hashes.SHA1())
18
19 expire_date = datetime.datetime.utcnow() + datetime.timedelta(minutes=30)
20
21 signer = CloudFrontSigner(KEY_ID, rsa_signer)
22
23 signed_url = signer.generate_presigned_url(
24     URL,
25     date_less_than=expire_date
26 )
27
28 print(signed_url)
```

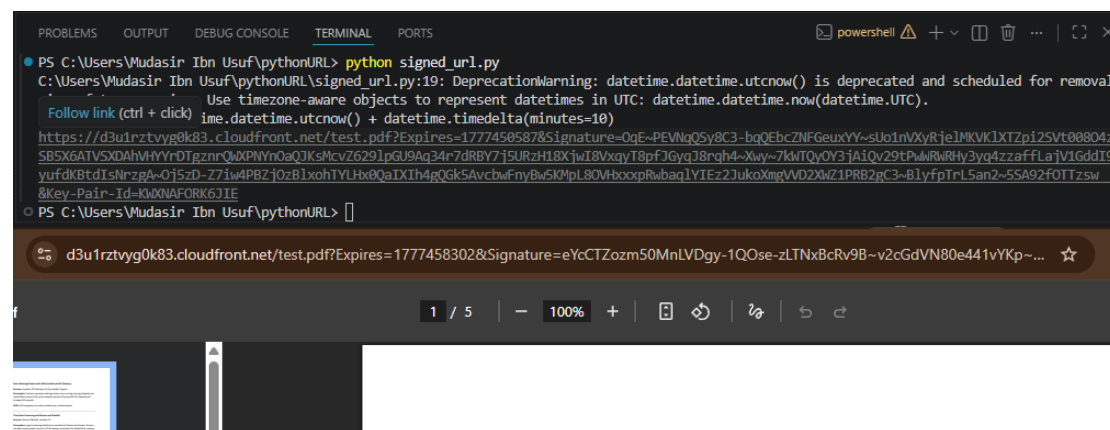
```
1 -----BEGIN RSA PRIVATE KEY-----
2 MIIEowIBAAKCAQEAh1eip2aV0i/hrZNBK8-Uw7F410780115oy5GCI9QU9VHGHI
3 ac+sC3v5oyY101f/g/APWqZq3V0LHf0411nFFPv/awCwW2Q58ngVLDm7M6/
4 iLDH64<+1rv1Lffjw8S8gRmq/sNG+eCt/vNultxfrPdltanUr412/gxg08mL6Z8
5 qJy5FahH2d0Hng6Q9+85qFrv37RgBoqfELf5M7rtysvsgdCoX9DfSS676jDsk
6 cTQc+sQEaQ35nqLX08D8Uow65Zr/0TTIF51hw11ow31b039BvzFqC/1BuDYF9ni
7 frpZedqr4P9H18a8tgZ2KcCT2pVQurySxvW1DAQA8A10Adnrb+2HARFLC38i
8 3JmLEFrZpmFanj1S8H7BclDmYwuh07JHhW7EEHUALT0L0mN/nJTEI+rv
9 eW8v0wYmLrhop85A7f16be7751111ZAF1brUhdRjqsZ939Yrf5ubkaQwNa
10 RTmAQ05VrPdRqFrem13dLte+v2eqfHClG6QhQJuh21937/3s1s08Q018X0H1
11 0w6F+ktEfV1s1AF0vX15o5CMV0w2mLwL074Z0mzYw0b15xTzldinihWdd1uRfN
12 TSg2oyzrATILQ0sRMEFYbnfPc4wCOLwL7Uuxgt958EDCP3PcHwaQg0vGdIEp
13 xVQ0WAEcgyEA66u1Wd0xu87qdeZaapwVhT7Z0Pe42tp9D6x4tQBuaP/3HRBK22
14 ZEp1o6Kn7FHLp8118P0wCug0htBwCf8qLxJfFOCJXVEpm0gg05Y88CD1H57q
15 0C+Ap08eII0czn0e0UezSteyneF3c54k16yNht0Xy0yHHTFxcP6A1Jg0
16 3Rzi52+xd0o0L586wQ8M5p0dy0vU00Et+F1l899var07FR315fLDTpCa
17 DG6E/U0h+T9w8h8WRE30JMerJLy3yEOCVc11K6Sm05t8BwEfpKL8bvcFED
18 JvJ91IEw1SyfFigY01QInRc9UNE0BU+CLYSeCgyEALWYmVhTubRdG5/yNm
19 PyZ1AS-cjYQ90mbus9UJE1UaL1kaEkc1Wq88VtGHQ8WmctKvJKPfu75sAX70k
20 XfAo5Pe0ft7q4MwZ5ZnpS1syppel+nr/b11Qpt28XakbMeOinPUChPnZHVUT1
21 0wJMPy/SVtupA31Y/vcu18Cgy8dg1eJ6a4A0d5QcVggCTG05v5GwD015rv
22 A49FR6+0yXQ8A+36d7p0c4VtL89R01h1pabombZ7XdyG6sP4W4hKqY
23 1SLK+Jr2R0Q9NTE3U1H0Pxt333IZ0MEKcPjgcFZTULmCJz0hJkrV2P8sX56
24 +h3FQK8g8e9wVt0415EKVWVaaQ0N2KmW4h1dHyCTdmdCLtV0Z6en0970A
25 3QJTuUum1186/00ziqeyTm6wJQ51btAcapH2Qd1n11bbEag3GLDC0LVY
26 1zweJ81RvTiVu0y77/GE50MPZzncazP9UqWEX3AC3M2P2bqmH
27 -----END RSA PRIVATE KEY-----
```

Step 14: Run Script

`python signed_url.py`

Output:

Signed URL generated



```
PS C:\Users\Mudasir Ibn Usuf\pythonURL> python signed_url.py
C:\Users\Mudasir Ibn Usuf\pythonURL> python signed_url.py: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal
Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
https://d3u1rztvyg0k83.cloudfront.net/test.pdf?Expires=1777458587&Signature=0qE~PEVnQ5Sy8C3-bqQEbCZNFGeuxYY~sUoInVxyRjeIMKVKLXTZpi25Vt08804z
S85X6ATVSXDAHVHYrDZgnrQXPNYN0aQJKsMcv6291pGU9Aq34r7dRBY715URzh18XjwI8VxqyT8pfJGyqJ8rqh4~Xuy~7kwtQvOY3JAiQv29TPwLRWRHy3yq4zafLaTVIGddT9
yufdKBtdIsNrzpA~Oj5zD~Z7iW4PBZjOzB1xohTYLHx0QaIXIh4gQGk5AvchwFnyBwSKMpL80Vb0cxpRwbaqLYIEz2JukoXmpVVD2XWZ1PRB2gC3~81lypTrL5an2~5SA92f0TTzsw
&Key-Pair-Id=K3WNAFORK6JIE
PS C:\Users\Mudasir Ibn Usuf\pythonURL>
```

<https://d3u1rztvyg0k83.cloudfront.net/test.pdf?Expires=1777458302&Signature=eYcTzozm50MnLVDgy-1QOse-zLTNx8CrV9B~v2cGdVN80e441vYKp~...>

Testing & Validation

Step 15: Test Access Behavior

Tested different access methods:

- S3 direct URL → Access Denied
- CloudFront normal URL → Access Denied
- Signed URL → Successfully accessed file

Purpose:

This confirms that only signed URLs are allowed and all other access paths are blocked

Step 16: Expiry Test

- Wait for expiration time

Expected: Access Denied

Purpose: Ensures temporary access control.

Security Behavior

- Signed URL can be accessed by anyone who has the link
- Access is limited by expiration time
- After expiry, access is automatically denied
- No direct access to S3 or CloudFront without signature

Troubleshooting

Issue 1: Access Denied via CloudFront

Solution:

- Check file name
- Verify bucket policy
- Ensure OAC is attached

Issue 2: Signed URL Not Working

Solution:

- Verify Key ID
- Check private key format
- Ensure correct CloudFront URL

Issue 3: File Not Found

Solution:

- Match exact file name
- Avoid spaces or encode URL

Lambda@Edge can be integrated to perform additional validation such as user authentication, IP restriction, or token verification before allowing access to CloudFront content.

Conclusion

This project successfully implements a secure file-sharing system using AWS S3 and CloudFront. Files are stored privately in S3 and accessed only through CloudFront using signed URLs. The use of Origin Access Control and IAM bucket policies ensures that direct access is blocked. Signed URLs provide temporary and secure access, making this architecture suitable for real-world applications such as document sharing and media delivery.